



윤경은

GenON FDE Engineer Intern

yge0307@gmail.com
github.com/ykgstar37-lab
yge-portfolio.vercel.app

고객 환경의 AI 병목을 백엔드·RAG·배포 구조로 해결하는 FDE 지향 엔지니어

핵심역량

- **검색/RAG 병목 진단** — PyMate(Team 4)에서 RAG 평가·임베딩 교체·Flask→Django 전환·AWS EC2 배포 4개 영역 단독 담당. RAGAS로 검색·생성을 분리 측정해 LLM 교체가 아닌 **embedding 차원 부족**으로 원인 확정 → Context Precision **0.83→0.97**, Recall **0.70→0.79**
- **데이터 파이프라인·라우팅 설계** — Seoul Culture Map(Personal)에서 공공 API 2종(좌표·카테고리 코드 불일치) 정규화 후 Intent→Retrieve→Generate **3-node** 라우팅을 단독 설계. 모든 요청을 단일 GPT-4o로 처리하는 baseline 대비 라우팅 적용 후 요청당 **\$0.003**, **85% 절감**
- **고객 환경형 AI 기능 안정화** — WorkFlow Agent(Team 4)에서 판단 Agent·4종 Guardrail·LoRA 데이터셋·vLLM 서빙 4개 모듈 단독 담당. 규정 도메인 GT QA **200건** LLM-as-a-judge 평가 기준 정확도 **37%→85%**, JSON 유효율 **70%→97%**, 과신 confidence **0.92→0.78** 보정
- **Python 백엔드 end-to-end** — 7개월간 3개 프로젝트에서 FastAPI/Django, **REST 11종**, SSE 스트리밍, Docker 서빙, AWS EC2 배포까지 단독 수행. 운영 환경 Nginx 프록시 타임아웃·Gunicorn 워커 설정으로 502 응답을 추적·복구

EDUCATION

가천대학교 응용통계학과

2020.03 ~ 2026.08 (졸업 예정)

SK Networks Family AI Camp

2025.09 ~ 2026.03 · AI/ML 엔지니어 집중 과정 · 팀 4명
RAG·Agent 프로젝트 2건 (PyMate·WorkFlow Agent)

CERTIFICATES

SQLD

한국데이터산업진흥원

ADsP

한국데이터산업진흥원

SKILLS

AGENT / LLM (직접 구현)

LangGraph LangChain OpenAI API vLLM LoRA
PyTorch

RAG / EVAL (직접 구현)

RAGAS Qdrant ChromaDB HyDE+BM25+RRF

BACKEND / INFRA

Python FastAPI Django PostgreSQL SQLite
Docker Nginx AWS EC2 SSE

DATA / STATISTICS

pandas scikit-learn numpy GARCH K-means
PCA SQL

SELECTED IMPACT

RAG QUALITY
PYMATE

DATA PIPELINE
SEOUL CULTURE MAP

AGENT RELIABILITY
WORKFLOW AGENT

0.83 → 0.97

RAGAS 기반 병목 진단

LLM 교체보다 먼저 검색과 생성을 분리
측정해 실제 병목을 확인했습니다.

85% ↓

3-node Intent Routing

단일 GPT-4o 호출(baseline) 대비,
일상 대화·검색을 분기해 호출량 자체를
줄인 결과입니다.

37% → 85%

4중 Guardrail

+ Confidence 보정

GT QA 200건 LLM-as-a-judge 평가
기준. 환각 응답을 서비스 노출 전에
차단할 검증 게이트를 직접
설계했습니다.

PROJECTS

PyMate · RAG 학습 챗봇

Team 4명 · 2026.01 ~ 02 · 6주

[GitHub](#) · 단독 담당: RAG 평가, 임베딩 교체, Flask→Django 전환, AWS EC2 배포 (4개 영역)

팀 4명 중 RAGAS 지표 정의·임베딩 차원 교체 의사결정·프로덕션 마이그레이션·운영 환경 디버깅은 본인이 단독 수행한 영역입니다.

"답변이 부정확하다"는 증상을 LLM 탓으로 돌리지 않고, RAGAS로 검색·생성을 분리 측정해 **embedding 병목**으로 재정의한 프로젝트입니다.

문제: 검색과 생성 단계가 섞여 있어, 더 비싼 LLM으로 바뀌도 실제 원인을 놓칠 위험이 있었습니다.

해결: RAGAS로 **Context Precision 0.83** 병목을 확인한 뒤, 임베딩을 **768D→3072D**로 교체하고 HyDE+BM25+RRF+Reranker 9단계 검색 구조를 재설계했습니다.

결과: **Precision 0.83→0.97, Recall 0.70→0.79**로 검색 품질을 끌어올렸습니다. 프로덕션 트래픽을 가정한 Django 마이그레이션 후 AWS EC2 배포에서 발생한 502 응답을 **Nginx 프록시 타임아웃·Gunicorn 워커 설정**으로 추적해 안정화했습니다 — 고객 환경에서 동일한 운영 이슈가 반복된다는 가정하에 디버깅 절차를 문서화했습니다.

배운 점: RAG 진단으로 얻은 "원인을 LLM 탓으로 돌리기 전 검색·생성을 분리 측정한다"는 절차는, WorkFlow Agent에서 LoRA v2 정확도 -3.2%p 하락 때 모델 교체가 아닌 오답 로그 19건 재분석·v3 학습 데이터 재구성으로 이어졌습니다.

[RAGAS](#) [Qdrant](#) [Django](#) [Nginx](#) [AWS EC2](#)

Seoul Culture Map · 서울 문화행사 Intent 라우팅 챗봇

Personal · 2026.03 ~ Present · Prototype

[GitHub](#) · Personal · 문제 정의·공공 API 정규화·라우팅 구조·API·UI·실행 환경까지 전 과정 단독 진행

서울시 문화시설 **2,500+**개 데이터를 일회성 보고서로 끝내지 않고 사용자가 직접 탐색·추천받을 수 있는 챗봇으로 확장하면서, 모든 요청을 단일 LLM에 태우는 비용 구조를 **Intent 라우팅**으로 해결했습니다 — 고객 환경에서 공공 데이터·API를 다루는 FDE 직무와 같은 결의 작업입니다.

문제: 분석 결과가 문서로만 남아 재사용되지 못했고, 단순 인사와 실제 검색 질의가 같은 LLM 파이프라인을 타며 비용이 커졌습니다.

해결: **Intent→Retrieve→Generate 3-node**로 분리해 일상 대화는 검색을 스킵하고, 검색형 요청만 SQL+ChromaDB RAG로 보내도록 설계했습니다.

결과: 로컬 임베딩과 SSE 스트리밍을 적용해 요청당 **\$0.003**(단일 GPT-4o baseline 대비 토큰 사용량·운영 비용 **85% 절감**), 첫 토큰 **~500ms**, 응답 완료 평균 **~2.4s**, 세션 문맥 **10턴**을 확보했습니다.

배운 점: 효율적인 Agent 설계는 무조건적인 고성능 모델 호출보다 의도 파악(Intent Routing)을 통한 최적 경로 설정이 비용·성능 트레이드오프의 핵심임을 체득했습니다. 고객 환경별 트래픽 분포가 달라질 경우 분기 룰을 데이터 기반으로 재조정해야 한다는 운영 관점도 함께 얻었습니다.

[OpenAI API](#) [LangGraph](#) [ChromaDB](#) [FastAPI](#) [SSE](#) [SQLite](#) [React](#)

WorkFlow Agent · 사내 업무 보조용 Agent 시스템

Team 4명 · 2026.02 ~ 04 · 8주

[GitHub](#) · 단독 담당: 판단 Agent, 4중 Guardrail, LoRA 데이터셋, vLLM 서빙 (4개 모듈)

팀 4명 중 검증 게이트 통과 정책·LoRA 데이터셋 설계·vLLM 서빙 구조는 본인이 의사결정·구현을 단독 수행한 영역입니다.

sLLM이 존재하지 않는 규정을 근거처럼 말하면서 confidence **0.92**를 주는 상황을 **검증 구조 문제**로 재정의한 프로젝트입니다.

문제: 규정 판단 도메인에서 오답·환각·과신이 함께 발생해, 고신뢰 오답이 서비스 레이어를 통과할 위험이 있었습니다.

해결: 판단 Agent에 **4중 Guardrail**과 **Confidence 보정**을 넣고, GPT/Claude/vLLM을 provider 패턴으로 교체 가능하게 통일했습니다.

결과: 규정 도메인 GT QA **200건** LLM-as-a-judge 평가 기준, LoRA **v1→v3 3회 이터레이션**으로 정확도 **37%→85%**, **JSON 유효율 70%→97%**를 확보하고 과신 confidence를 **0.92→0.78**로 보정했습니다.

배운 점: v2에서 **98건** 추가 학습 후 정확도 **-3.2%p** 하락을 확인하고, 오답 로그 **19건**만 다시 설계해 v3로 수정했습니다. Agent 신뢰성은 모델 성능보다 실패 처리 구조까지 함께 설계해야 안정화된다는 기준을 얻었습니다.

[LangGraph](#) [vLLM](#) [LoRA](#) [Guardrail](#) [FastAPI](#) [Qdrant](#) [Docker](#) [RunPod A100](#)

STATISTICS FOUNDATION

지표 정의 → 구조 변경 흐름

응용통계학 전공으로 가설·실험 설계 기본기를 익힌 뒤, 7개월간 3개 프로젝트에서 정확도·Precision/Recall·JSON validity·confidence·비용·latency **6종 지표**를 먼저 정의하고 구조를 바꿨습니다. RAGAS·LLM-as-a-judge로 측정 가능한 형태를 만든 뒤 의사결정에 들어가는 절차를 유지합니다.

실패 실험을 다음 판단 기준으로

GARCH·회귀·군집·PCA 분석 경험을 바탕으로, WorkFlow Agent에서 LoRA v2 정확도 **-3.2%p** 하락을 데이터 노이즈 가설로 해석하고 v3에서 오답 로그 **19건**만 재학습해 회복했습니다. PyMate에서도 LLM 교체 가설을 RAGAS Context Precision으로 기각하고 임베딩 차원 교체로 방향을 바꿨습니다.

WHAT I BRING TO GENON FDE

무엇을 가지고 가는가

"답변이 부정확하다"는 모호한 증상을 RAGAS로 검색·생성을 분리 측정해 **embedding 차원 부족**으로 좁혀 낸 진단 경험과, sLLM 과신 응답을 GT QA 200건 평가 기준 4중 Guardrail로 차단해 본 검증 게이트 설계 경험. 고객 환경에서 "AI가 이상하게 답한다"는 보고를 받았을 때, 가장 먼저 어디를 측정해 분리할지 절차로 가지고 있습니다.

어디서 보탬이 되는가

FastAPI/Django 백엔드, 공공 API 정규화, RAG 9단계 검색 구조 재설계, AWS EC2 Nginx·Gunicorn 운영 디버깅 경험을 바탕으로 — **고객 환경 연동·데이터 파이프라인·검색/RAG·운영 안정화** 흐름에서 PoC를 측정 가능한 형태로 옮기고 운영 이슈를 절차화하는 작업에 손이 빈 영역을 메우려 합니다.